Embedded C Style – Manual

Mohamed Yaqoob

November 2021

This manual summarises a collection of C programming Style Rules and Good practices known within the embedded community, and specifically the convention that we like to use here at Mutex Embedded. This manual forms an easy-to-use daily reference for developers.

COPYRIGHT NOTICE

Table of Contents

1. C Language

Rule 1.1 – Use C99 Standard

- You can use GNU99 mode if you require extended C features such as **atoff().**

Rule 1.2 – Make use of the C standard libraries

There are many C Standard libraries that you can use instead of writing code from scratch. The following are the most common standard libraries and their brief description:

<complex.h> Complex number arithmetic

<ctype.h> Characters types checking

<limits.h> Sizes of basic types

<math.h> Common mathematics functions

<stdarg.h> Variable arguments capture

<stdbool.h> Boolean type

<stdint.h> Fixed-width integer types

<stdio.h> Input/output

<stdlib.h> General utilities such as: Memory, program, string conversions, random and algorithms

<string.h> String handling library

<time.h> Time and date utilities

2. Code Format

Indentation

- Avoid using the tab \t character across your entire code.
- Using spaces only for indentation.
- Indentation size is 2 spaces
- Indent statements within **function** body
- Indent statements within **block**
- Indent statements within **switch** body
- Indent statements within **case** body
- Indent **break** statements
- Indent **labels**

Braces

- Brace on the **next line** for **function** declaration
- Brace on the **next line** for **blocks**
- Brace on the **next line** for **blocks in case** statements
- Brace on the **next line** for **switch** statement
- Brace on the **next line** for **linkage** (e.g. extern)

White Space

- Add white space for **Declarator list** after comma and after pointer
- For **functions**, no white space before pointer, add a white space after pointer
  e.g. void updateValue (int* pValue);
- Add a white space before opening parenthesis for **function**
- Add a white space after semicolon in **for-loop** parameters
- Add a white space after comma in **function arguments**
- Add a white space before and after **assignment and binary operators**
- Add a white space after comma in **Initializer list**

Control Statements

- Insert new line before **else** in an **if** statement
- Do not insert new line before **while** in a **do** statement
- Keep **else if** on the same line

Line Wrapping

- A maximum line width of 120 characters is recommended.
- Indent wrapped line 2 indentation levels
- For wrapped functions, wrap all elements each on a new line
- Enum list always wrapped regardless of line width
- For wrapped expressions, wrap only necessary parts
- For wrapped initializer list, wrap only necessary parts

3. Variables

Rule 3.1 – Use camelCase naming convention

- First word starts with a lower case and all following words start with upper case

```
int16_t initialVerticalSpeed;
float gpsAverageAltitude;
```

Rule 3.2 – Use clear and descriptive names

- Avoid using abbreviations
- Make the variable name as short as possible, yet clear.

```
int16_t avgAlt;          //(Bad)
int16_t averageAltitude; //(Good)
```

Rule 3.3 – Declare variables in separate lines

- This generally improves readability.
- It reduces future potential bugs related to multiple variables in a single line, like delete or comment out.

```
uint32_t startTime, finalTime, deltaTime;   //(Bad)
uint32_t startTime;   //(Good)
uint32_t finalTime;
uint32_t deltaTime;
```

- Additionally: **Align variable names and values for a group of declarations**, for better readability, as follows:

```
uint16_t totalGreenApples = 15;
uint16_t totalRedApples   = 17;
float    verticalSpeed    = 12.56f;
```

Rule 3.4 – For Pointer variables, use the letter 'p' as a prefix

- No space shall be placed between type and pointer * symbol, a space is added to the variable name, as follows:

```
char* pStudentName;
```

Rule 3.5 – For Double Pointer variables, use "pp" as a prefix

```
char** ppWifiNames;
```

Rule 3.6 – For Global variables, use the letter 'g' as a prefix

```
static uint8_t gNextIndex;
```

Rule 3.7 – When a variable is both Global and Pointer, use pointer prefix

Example 1 – Global scope: A Pointer → Use global prefix

```
static char* gLastName;
```

Example 2– Global scope: Normal variable → Use global prefix

```
static uint8_t gNextIndex;
```

Notes:

- Local scope: Referring to when a variable is declared within a function
- Global scope: Referring to when a variable is declared globally in a source file, or a more special case is when declared as static within a function.

Rule 3.8 – Use Static keyword when a variable or function is to be visible only within a single source file

- It is recommended to keep variables within a source file static if they are not meant to be visible by other files.
- However, it is **not recommended to define a static variable within a function**, this can make your code less readable and more susceptible to bugs.

Rule 3.9 – When a variable is used by interrupt routine or is a memory-mapped pointer, it must be declared as volatile.

- Tick counters are incremented in an Interrupt routine, thus volatile
- Memory-mapped pointers values are volatile since a register can be changed by hardware.

Example 1 – Tick counter

```
volatile uint32_t gCounterTicks = 0;

void isrIncrementCounter (void)
{
  gCounterTicks++;
}

uint32_t getCounter (void)
{
  return gCounterTicks;
}
```

Example 2 – Memory-mapped pointer

```
volatile uint8_t* pInputSwitch = (uint8_t*)(GPIOA->IDR);

while(!(*pInputSwitch & 0x01)); //Wait indefinitely for switch
printf("Switch is Activated!");
```

Rule 3.10 – Typedef your application variable whenever possible

- It is generally recommended to typedef all variables in your application to a user typedef.
- Using the raw c types in application is not recommended as changing the type later is way more complex than using typedef.
- Include unit name as part of variable type name when applicable, as follows:

```
typedef uint32_t GpsLatitude_Micro_t;
typedef float    AverageSpeed_Kmh_t;
typedef uint32_t Distance_Meters_t;
```

4. Const function parameters

At this section, we are only going to focus on the use of the **Const** keyword for passing parameters to functions.

Rule 4.1 – For constant value parameter, use the constant keyword for the variable or type only, as follows:

```
void setSpeed (const uint16_t speed)
{
  gMotorConfig->forwardSpeed = speed;
}
```

Rule 4.2 – For constant pointer parameter, use the const keyword to operate on the pointer only, as follows:

```
void updateWifiNameTo4G (WifiName_t* const pWifi)
{
  sprintf(pWifi->buf, "%s - 4G", pWifi->buf);
}
```

Rule 4.3 – For both constant value and constant pointer parameter, use the const keyword for both the type and the pointer, as follows:

```
void printWifiName (const WifiName_t* const pWifi)
{
  printf("Selected Wifi Name = %s\n", pWifi->buf);
}
```

5. Structure

Rule 5.1 – Always typedef a structure

```
typedef struct
{
…
}
```

Rule 5.2 – Use PascalCase for Structure name

```
typedef struct
{
…
}AccelData_t;
```

- Typedef variable name still uses camelCase

```
AccelData_t accelData;
```

Rule 5.3 – Add the suffix _t to the typedef name

```
typedef struct
{
  int16_t accelX;
  int16_t accelY;
  int16_t accelZ;
  bool    isValid;
}AccelData_t;
```

Rule 5.4 – Use filename as a prefix to the struct typedef name

```
typedef struct
{
  int16_t accelX;
  int16_t accelY;
  int16_t accelZ;
  bool    isValid;
}Mpu6050_Accel_t;
```

Rule 5.5 – Do not memory-map structure bitfields, instead access them directly

```
typedef struct
{
  uint8_t reserved : 3; //LSb
  uint8_t afs_sel : 2;
  uint8_t za_st : 1;
  uint8_t ya_st : 1;
  uint8_t xa_st : 1;    //MSb
}Mpu6050_AccelConfigReg_t;

int main (void)
{
  Mpu6050_AccelConfigReg_t hAccelConfig;
  hAccelConfig.xa_st = 0;
```

```
  hAccelConfig.ya_st = 1;
  hAccelConfig.za_st = 0;
  hAccelConfig.afs_sel = 2;
  hAccelConfig.reserved = 0;

  uint8_t accelConfigBits = *(uint8_t *)&hAccelConfig;

  getchar();
  return 0;
}
```

6. Enums

Rule 6.1 – Always typedef an Enum

```
typedef enum
{
…
};
```

Rule 6.2 – Use PascalCase for Enum Name
Rule 6.3 – Add the suffix _e to the Enum name

```
typedef enum
{
…
}FullScale_e;
```

Rule 6.4 – Enum elements name shall start with the prefix <TypedefName_>

```
typedef enum
{
  FullScale_2g = 0,
  FullScale_4g,
  FullScale_8g,
  FullScale_16g,
}FullScale_e;
```

Rule 6.5 – Use filename as a prefix to Enum name and hence the individual elements

```
typedef enum
{
  Mpu6050_FullScale_2g = 0,
  Mpu6050_FullScale_4g,
  Mpu6050_FullScale_8g,
  Mpu6050_FullScale_16g,
}Mpu6050_FullScale_e;
```

7. If-Condition

Rule 7.1 – Avoid using single line condition

- Always use multiple line body and enclose with braces

```
if(accelZ > 800)
{
  printf("Mostly Level, Z-Accel = %d mg\n", accelZ);
}
```

Rule 7.2 – When checking for multiple conditions, enclose conditions with parentheses

- Use Boolean conditional operators for multiple conditions relationship such as (OR: ||, AND: &&)

```
if((verticalSpeed > 12) && (horizontalSpeed < 5))
{
}
```

Rule 7.3 – When comparing a variable with constant, put constant first on the left-hand side

- When you forget to put double equals =, compiler will detect an assignment to constant error when the constant is on the Left Hand Side, which can be very useful.

```
if(12 == verticalSpeed)
{

}
```

8. Switch-Case

Rule 8.1 – Use Enum as switch case variable, whenever possible

- Avoid using generic types such as integer because they take on so many values.

Rule 8.2 – Do not put a default case when an Enum is used as the switch case variable

- This allows compiler to catch missing or unhandled cases, and throw warning.

```
Mpu6050_AccelFullScale_e fullScaleSelect = Mpu6050_AccelFullScale_4g;

switch(fullScaleSelect)
{
  case Mpu6050_AccelFullScale_2g:
    break;

  case Mpu6050_AccelFullScale_4g:
    break;

  case Mpu6050_AccelFullScale_8g:
    break;

  case Mpu6050_AccelFullScale_16g:
    break;
}
```

Rule 8.3 – To define a new variable within a case, add a code block explicitly

- Cases are considered labels in C and thus do not support variable definition by default.
- Note: It is recommended to keep the **break** keyword inside the code block, for better readability.

```
  case Mpu6050_FullScale_4g:
  {
    int value = 4;
    printf("FS %dg\n", value);
    break;
  }
```

Rule 8.4 – If multiple cases have the same handling, use fall through.

- Fall through is enabled by writing cases in series with just a single break at the end and a common body for all.

```
  case Mpu6050_FullScale_8g:
  case Mpu6050_FullScale_16g:
  case Mpu6050_FullScale_32g:
    printf("FS 8g or higher\n");
    break;
```

Rule 8.5 – When using Non-Enum switch case variable, default case must be added

- This is simply because integer variable can take on many different values, unlike Enum where elements are bounded.

```
int mySelect = 1;
switch(mySelect)
{
  case 1:
    break;

  case 2:
    break;

  default:
    break;
}
```

9. For-Loop

Rule 9.1 – Avoid using magic numbers for loop counter

- Use define constants instead.
- When the constant is changed, all instances will be updated unlike magic numbers where you need to change them individually.

```
#define ELEMENTS_COUNT  10

int main (void)
{
  for(uint8_t i = 0; i < ELEMENTS_COUNT; i++)
  {
    printf("Counter = %d\n", i);
  }
  getchar();
  return 0;
}
```

10. Macros

Rule 10.1 – Use full upper-case letters for Macros

- Separate words with underscore _

```
#define MAX_HEIGHT  100
```

Rule 10.2 – When defining a series of defines, use double underscore __ to separate prefix text from individual elements name

```
#define MAIN_TASK_SIG__SUSPEND        os_SignalNum_0
#define MAIN_TASK_SIG__RESUME         os_SignalNum_1
#define MAIN_TASK_SIG__HEAP_MONITOR   os_SignalNum_2
#define MAIN_TASK_SIG__RUN_STATS      os_SignalNum_3
```

Rule 10.3 – Enclose arithmetic operations within Macros with parentheses

```
#define MAX_HEIGHT  (50 * 2)
```

Rule 10.4 – For parametric or function-like Macros, surround any use of Macro parameters with parentheses

- This will ensure that parameters arithmetic operations are resolved first

```
#define TEMPERATURE_CONVERT__C_TO_F(X)  (((X) * 1.8f) + 32.0f)

printf("46.30C = %.2fF\n", TEMPERATURE_CONVERT__C_TO_F(45.3 + 1.0f));
```

Macro Expansion:

```
---> (((45.3 + 1.0f) * 1.8f) + 32.0f)
```

11. X Macro

At this section, we will demonstrate how to correctly use X Macros. We will demonstrate this with automatically typing and expanding the following code:

```
enum
{
  DatabaseId_BLDC_Speed,
  DatabaseId_BLDC_Status,
  DatabaseId_BLDC_MotorRpm,
}VariablesIds_e;

void     setBldcSpeed    (uint16_t bldcSpeed);
uint16_t getBldcSpeed    (void);
void     setBldcStatus   (bool bldcStatus);
bool     getBldcStatus   (void);
void     setBldcMotorRpm (float bldcMotorRpm);
float    getBldcMotorRpm (void);
```

Where, we are going to create an Enum that populates automatically with the X Macro, and create Set/Get functions for each variable.

Step 1 – Create the LIST Macro

- This is the only Macro that user can interact with to add and remove elements.

```
#define DATABASE_LIST \
  DATABASE_DATA(DatabaseId_BLDC_Speed, uint16_t, bldcSpeed) \
  DATABASE_DATA(DatabaseId_BLDC_Status, bool, bldcStatus) \
  DATABASE_DATA(DatabaseId_BLDC_MotorRpm, float, bldcMotorRpm)
```

Step 2 – Create the APPLICATION Macros

- APPLICATION macro refers to the user-specific macro.
- At this case, we will add 2 Macros: 1) Enum elements, 2) Set/Get functions
- This makes use of the LIST macro to create dynamically expanding code.

Enum elements

```
typedef enum
{
#define DATABASE_DATA(row_id, type, name) \
  row_id,
  DATABASE_LIST
#undef DATABASE_DATA
}VariablesIds_e;
```

Set/Get functions prototypes

```
#define DATABASE_DATA(row_id, type, name) \
  void set##name (type name); \
  type get##name (void); \

  DATABASE_LIST
#undef DATABASE_DATA
```

Main code

```
setbldcSpeed(123);
printf("BLDC Speed = %d\n", getbldcSpeed());
```

For further explanation of the X Macro, watch my **YouTube** video on X Macro as part of the Embedded C Programming Style tutorial series.

12. Functions

Rule 12.1 – Function name shall follow the camelCase convention

```
uint16_t getAccelX (void);
```

Rule 12.2 – Function name shall be clear and concise

- Name needs to be small, yet descriptive of its purpose.

Rule 12.3 – For library functions, use the library name as a prefix

- Use an underscore to separate the prefix from function name as follows:

```
uint16_t mpu6050_getAccelX (void);
```

Rule 12.4 – For callback functions, start the function name with "cb"

```
void mpu6050_cbBarachuteOpened (void)
{
  //Do something...
}
```

Rule 12.5 – For Boolean conditional functions, start the function name with "is"

- This does not apply to all Boolean returning functions, rather for functions that explicitly perform a Boolean checking operation.

```
bool osLayer_isInIrqHandler (void);
```

Rule 12.6 – Functions shall have a single exit point

- That is, functions shall have a single **return** statement whenever possible.
- Having a **single return** helps reducing code complexity and makes debugging easier.

Multiple returns (Not preferred)

```
bool mpu6050_readAccelX (uint16_t* pValue)
{
  uint16_t accelDataRaw;
  if(!mpu6050_i2cReadData16(0x00, &accelDataRaw))
  {
    return false;
  }
  if(!mpu6050_isAccelDataRangeValid(accelDataRaw))
  {
    return false;
  }
  //Scale by 1/1000;
  *pValue = accelDataRaw * 0.001f;
  return true;
}
```

Single return (Good)

```
bool mpu6050_readAccelX (uint16_t* pValue)
{
  bool isSuccess = false;
  uint16_t accelDataRaw;
  isSuccess = mpu6050_i2cReadData16(0x00, &accelDataRaw);
  if(isSuccess)
  {
    isSuccess = mpu6050_isAccelDataRangeValid(accelDataRaw);
  }
  if(isSuccess)
  {
    //Scale by 1/1000;
    *pValue = accelDataRaw * 0.001f;
  }
  return isSuccess;
}
```

Rule 12.7 – Do not use standard C library functions names

- E.g. printf, scanf, memcpy, etc…
- If you have a function with similar properties, try naming it differently or use a distinct prefix. (e.g. **printf → userPrint()** or **user_printf()**)

13. Header and Source files

Rule 13.1 – Use lower-case letters for file names

- This is important for case-insensitive file systems.
- For multiple words file name, use underscore "_" to separate words (e.g1. **usb_device.h**, e.g. 2 **log_print_io.h**)

Rule 13.2 – Make file name unique, clear, and as small as possible

- File name needs to be distinct and not confused with other files
- It needs to be clear and descriptive of its purpose yet uses minimum characters or words.

For example, I have a file that manages clock events, a good name for it will be as follows:

clock_events.h

Additionally, I have another related file that handles clock events in lower-level, I can name it the same but with the suffix "ll" for low-level to make it unique:

clock_events_ll.h

Rule 13.3 – Make a template for Header and Source files

- It is generally a good practise to have a template for both Header and Source files for keeping file information and copyright notices, instead of writing that from scratch each time.

Rule 13.4 – Header file shall have a protection against multiple includes

- The entire Header file code shall be enclosed withing the following ifndef and endif:

```
#ifndef DATABASE_TEST1_H_
#define DATABASE_TEST1_H_
//Code goes here
#endif
```

- The #define NAME need to be unique to each header file, usually it uses the actual file name in upper-case letters (e.g. clock_events.h → #define CLOCK_EVENTS_H_).

Rule 13.5 – Header file shall not declare variables or allocate memory space

Rule 13.6 – Header file shall have minimum includes

- The rest of includes apart from those required by the header file, shall go into the source file.

Rule 13.7 – Header and Source file shall have ordered content

Header file

Header file content shall be ordered as follows:

- Header comment
- Multiple includes protection
- Includes
- Types
- Functions prototypes
- Callback prototypes (callback functions prototypes)

Source file

Source file content shall be ordered as follows:

- Header comment
- Includes
- Macros / Defines
- Variables
- Private functions prototypes
- Private functions definitions
- Public functions definitions
- Callback definitions

14. Copying code

Rule 14.1 – Only copy the re-usable part of code

- Never copy the full code with the intention to change it, rather copy only parts of the code that you need fully.

Here is an example of copying re-usable code safely. I have some Database library that has a set function for each variable. Currently it has Tank Level set function. Set Battery function needs to be added.

```
void databaseApp_gui_setTank (Database_GUI_TankLevel_t level)
{
  database_setUint8(DatabaseId_GUI_MAIN_Tank, level);
}
```

Step (1): Function name

```
void databaseApp_gui_set
```

Step (2): Put the right name

```
void databaseApp_gui_setBattery ()
```

Step (3): Copy parameter part

```
void databaseApp_gui_setBattery (Database_GUI_)
```

Step (4): Manually fill in name

```
void databaseApp_gui_setBattery (Database_GUI_Battery_t battery)
```

Step (5): Copy body line upto re-usable part

```
{
  database_set
}
```

Step (6): Manually fill in the remaining parts to get to this final form

```
void databaseApp_gui_setBattery (Database_GUI_Battery_t battery)
{
  database_setFloat(DatabaseId_GUI_MAIN_Battery, battery);
}
```

Rule 14.2 – If your library has many re-usable pieces of code, consider using X-Macro.

- Use of X-Macro can help you automate your code safely.
- Use of X-Macro is recommended when many parts of code are repeated.

15. Commenting

Rule 15.1 – Add a comment for non-obvious code or where additional information is necessary

- Avoid adding trivial comments, to make your code tidy and easy to read

```
//Accelerometer full scale, read page 15 of the Registers map manual
typedef enum
{
  Mpu6050_AccelFullScale_2g = 0,
  Mpu6050_AccelFullScale_4g,
```

Rule 15.2 – Use single line comments in general

Rule 15.3 – Use multiple line comment for commenting a section and single line comment for sub-sections

```
/* MPU6050 Initialisation sequence */
//Allocate memory for the object
Mpu6050_Config_t* pConfig = malloc(sizeof(Mpu6050_Config_t));
//Set the I2C address
pConfig->i2cAddrs         = addrs;
//Intialise the full scale value
pConfig->accelFullScale   = fullScale;
```

Rule 15.4 – To comment out a block of code, use #if 0 macro #endif

- Do not use actual comment to disable part of code, it is much safer to use #if 0 macro
- It generally improves readability

```
#if 0
void mpu6050_cbHighAltitudeDetected (uint16_t altitude);

bool mpu6050_isFreeFallDetected (void);

bool mpu6050_readLoadCurrent (float* pCurrent);
#endif
```

Rule 15.5 – Add TODO comment for any incomplete code

- Include developer name in the comment to help tracing changes "TODO (Mohamed Yaqoob): ..."

```
void mpu6050_updateAccelData (Mpu6050_Config_t* pConfig)
{
  //TODO(Mohamed Yaqoob): Link to I2C data read function
  pConfig->accelX = 20;
  pConfig->accelY = 40;
  pConfig->accelZ = 940;
}
```

Rule 15.6 – Use Doxygen for function documentation

- Doxygen can be used to generate documentation, especially for functions
- Put function comments or documentation in header file only

```
/**
 * @brief Get Accelerometer X-Axis data
 * @param pConfig
 * @return acceleration in milli-g unit
 */
int16_t mpu6050_getAccelX (Mpu6050_Config_t* pConfig);
```